2026 DESIGN AND VERIFICATION™ DVCON CONFERENCE AND EXHIBITION INDIA

September 1-3, 2026
Radisson Blu, Marathahalli, Bangalore

Edge-Optimized Task-Driven Detection:

A VEGA-FPGA Co-Design Using Semantic Pre-Filtering and YOLOE Nano

Team Sitting Ducks
Bimsara Udurawana
Tharaka Jayasena
Thilakshan Balakrishnan

Department of Computer Science and Engineering
University of Moratuwa.

Table of Contents

Abstract....4
I. Introduction....5
II. Background Research....5
III. Goal and Objectives....7
IV. Design Process....8
i. Problem Statement....8
ii. Functional Specification....8
iii. Proposed Design....9
iv. Analysis of Final Design....11
v. Testing and Implementation of Final Design....13
V. Results and Discussion....14
VI. Conclusion....14
VII. References....15

List of Tables

N/A

List Of Figures

Fig1: Diagram of proposed design of the model.
Fig2: Sample test results of early implementation attempts.
Fig3: Sample tests benchmarks

List of Symbols

YOLOE Nano: YOLOE-26N model

MiniLM : all-MiniLM-L6-v2 model

Abstract

Robotic autonomy requires vision systems that can understand open-ended human instructions. However, deploying massive Vision-Language Models (VLMs) on edge devices is computationally unfeasible due to strict power and memory limits. This project presents a low-power Hardware/Software Co-Design for the Genesys-2 board that solves this bottleneck by fundamentally decoupling semantic reasoning from visual processing.

Rather than deploying a monolithic VLM, our architecture fundamentally decouples semantic reasoning from parallel visual processing. The software layer, running on the VEGA RISC-V processor, utilizes the lightweight MiniLM language model to evaluate text prompts and dynamically generate a pre-filtered, strictly ranked list of target classes via zero-shot semantic matching. This vocabulary is transmitted to the FPGA, effectively blinding the hardware to irrelevant classes to conserve computational power.

The vision layer utilizes a custom Convolutional Neural Network (CNN) accelerator optimized for the YOLOE Nano backbone. By leveraging YOLOE's one-to-one bipartite matching, it eliminates the Non-Maximum Suppression (NMS) sequential bottleneck. To further satisfy the limitations of the edge device, the design implements INT8 weight quantization, row-pass line buffering and various other techniques to optimize performance.

Ultimately, this architecture successfully translates complex human instructions into precise bounding boxes by strictly adhering to hardware constraints

I. Introduction

The advancement of robotic autonomy requires intelligent vision systems capable of interpreting human tasks, not just recognizing rigid object categories. While large-scale Vision-Language Models (VLMs) bridge this semantic gap, their massive computational requirements make them unsuitable for resource-constrained edge devices.

This project proposes an efficient, low-power, open-vocabulary detection pipeline for edge robotics, specifically targeting the Genesys-2 board through a novel Hardware/Software Co-Design approach.

Rather than forcing a monolithic VLM onto edge silicon, we decouple semantic reasoning from visual processing. We utilize a lightweight language model (MiniLM) on the VEGA RISC-V processor to pre-filter task-appropriate target classes via zero-shot semantic matching. This dynamically programs a highly efficient, NMS-free vision backbone (YOLOE Nano), which is custom-accelerated on the onboard FPGA.

This asymmetric partitioning effectively translates complex human instructions into precise bounding boxes while strictly adhering to hardware constraints.

II. Background Research

The development of our proposed architecture is heavily informed by recent paradigm shifts in computer vision, natural language processing, and edge-hardware acceleration.

1. The Evolution of Open-Vocabulary Detection

Historically, object detection models like early YOLO iterations operated on a "closed-vocabulary" paradigm, requiring extensive retraining and annotation to identify novel objects outside their rigid training sets. Recently, Vision-Language Models (VLMs) like Apple's MobileCLIP and YOLO-World [1]

have revolutionized this space. By projecting images and text into a shared latent space, these models enable "open-vocabulary" zero-shot detection, locating objects based purely on semantic prompts.

2. The Semantic Gap in Task-Driven Robotics

While open-vocabulary models can identify a "cup," robotic autonomy requires mapping complex instructions (e.g., "serve a beverage") to the correct physical tool. Traditionally, Large Language Models (LLMs) bridge this semantic gap. However, deploying multi-billion parameter LLMs alongside heavy vision models on edge devices is computationally unfeasible due to strict power, thermal, and memory bandwidth limits.

3. Challenges in FPGA Edge Acceleration

Deploying AI on FPGAs like the Genesys-2 presents unique bottlenecks. While FPGAs excel at parallelized matrix multiplications, they struggle with sequential algorithms like Non-Maximum Suppression (NMS). NMS—a post-processing filter for overlapping bounding boxes—forces the FPGA to halt parallel processing and shuttle data to the CPU, causing severe latency spikes [2]. Furthermore, the massive memory footprints of modern VLMs overwhelm the FPGA's limited Block RAM (BRAM), requiring constant, power-hungry external DDR memory accesses.

Synthesis for Proposed Design

Our research indicates a critical need for an architecture that decouples these workloads. To achieve task-driven intelligence without exceeding edge constraints, the semantic reasoning must be offloaded to a highly efficient, CPU-friendly model (such as MiniLM) [3], while the vision backbone must utilize an NMS-free, hardware-optimized architecture (such as YOLOE Nano) [4] to maintain an uninterrupted, highly parallelized dataflow on the FPGA.

III. Goal and Objectives

Goal

The primary goal of this project is to design and implement a low-latency, task-driven object detection system on the edge-constrained Genesys-2 board.

Objectives

To accomplish this goal, the following specific objectives must be met:

1. **Semantic Pre-Filtering (Software):** Implement the lightweight MiniLM language model on the VEGA RISC-V processor to translate complex task prompts into a strictly ranked, dynamically filtered list of target classes.
2. **Vision Acceleration (Hardware):** Design a custom Convolutional Neural Network (CNN) hardware accelerator on the Genesys-2 FPGA specifically optimized for the YOLOE Nano vision backbone.
3. **Efficient Hardware/Software Communication:** Create a direct data link where the VEGA CPU tells the FPGA exactly which few objects to look for *before* analyzing the image. By sending this narrowed-down list, the FPGA saves significant power and time by only searching for items relevant to the user's prompt, rather than its entire 80-item catalog.
4. **Task-Appropriate Selection Logic:** Develop a strict semantic arbitration algorithm on the VEGA processor that evaluates the FPGA's raw visual detections against the language model's priority list, ensuring the system targets the most contextually appropriate object present in the frame.
5. **Resource Optimization:** Apply aggressive hardware optimizations, including INT8 weight quantization, image tiling, and memory pipelining within the FPGA's Block RAM (BRAM), to maximize DSP utilization and minimize power-hungry external memory accesses.

IV. Design Process

I. Problem Statement

Current edge AI vision systems are fundamentally limited by closed-vocabulary training, rendering them incapable of interpreting dynamic, open-ended human instructions (e.g., "find something to hold water"). While Vision-Language Models (VLMs) solve this semantic mapping problem, their massive architectural size, reliance on sequential processing, and high memory bandwidth requirements make them incompatible with the strict power, thermal, and spatial constraints of edge computing hardware. The engineering challenge is to design an open-vocabulary, task-driven detection pipeline that fits entirely within the resource budget of the Genesys-2 board, accurately bridging text prompts and static image inputs without relying on cloud computation.

II. Functional Specification

The system is designed to operate as a cohesive Hardware/Software Co-Design with the following specifications:

i. **System Inputs:**
1. A natural language text string representing a human task (e.g., "Serve a beverage").
2. A static image file passed to the board for visual analysis.

ii. **System Outputs:**
1. Precise spatial bounding box coordinates [x, y, w, h] of the targeted object.
2. The semantic class label of the selected object.

iii. **Performance Constraints:**

a. **Latency:** The end-to-end processing pipeline must be heavily optimized to minimize execution time per image, ensuring rapid task resolution without bottlenecking the VEGA CPU.
b. **Resource Utilization:** The custom CNN accelerator must strictly operate within the available DSP slices and Block RAM (BRAM) constraints of the Genesys-2 FPGA.
c. **Accuracy Mechanism:** The system must definitively output *only* the single most task-appropriate object present in the static frame, enforcing strict semantic arbitration over raw visual confidence.

III. Proposed Design

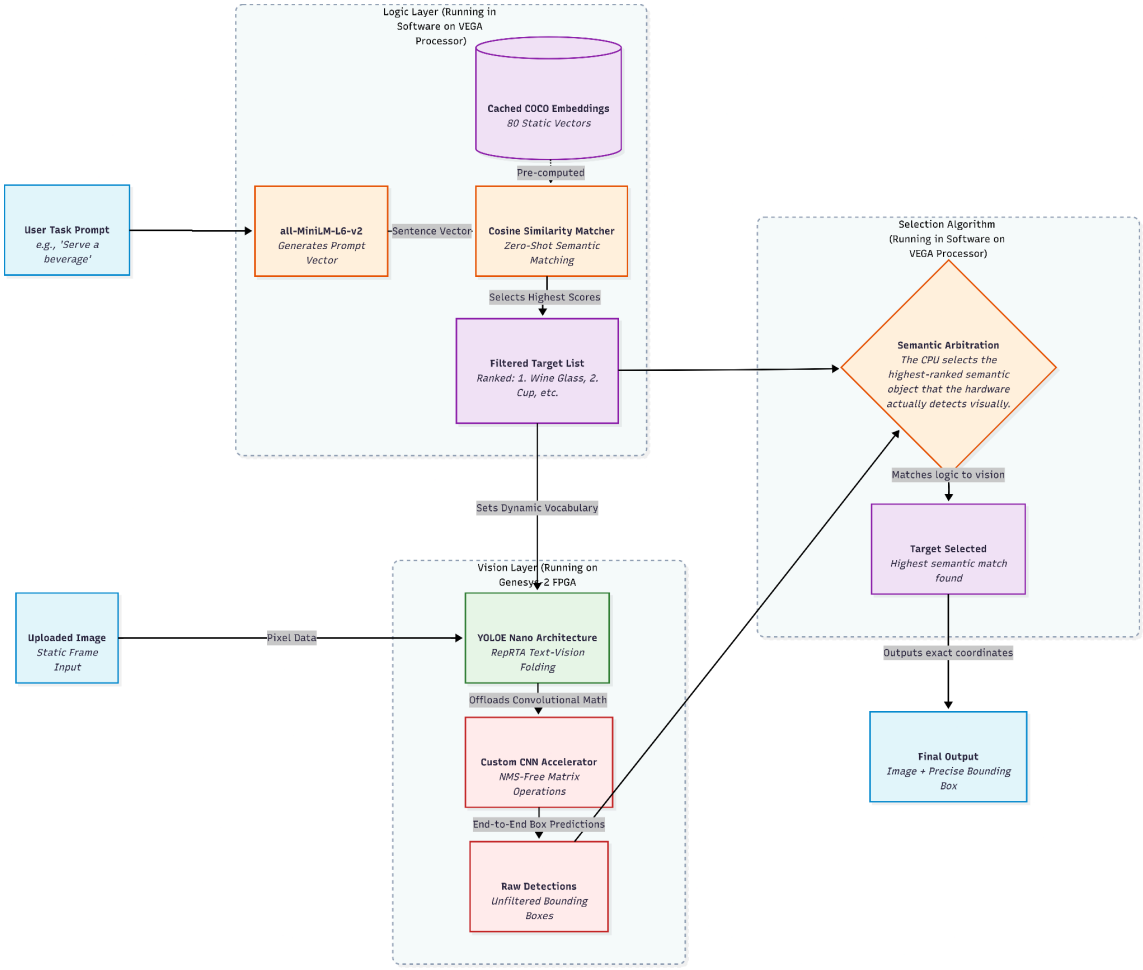

Fig 1: Diagram of proposed design of the model

To satisfy the strict resource constraints of the Genesys-2 board while achieving open-vocabulary task detection, our proposed architecture employs an asymmetric Hardware/Software Co-Design.

This pipeline fundamentally decouples sequential semantic reasoning from parallel visual processing, allocating workloads to the silicon best suited for them. The system operates in three distinct phases:

1. Semantic Pre-Filtering (VEGA Software Layer)

When a text prompt (e.g., "Serve a beverage") is input into the system, the VEGA RISC-V processor initializes the Natural Language Processing (NLP) pipeline. We utilize the lightweight MiniLM transformer model to generate a 384-dimensional mathematical embedding of the prompt.

The CPU then calculates the Cosine Similarity between this prompt vector and a locally cached database of the 80 standard COCO object embeddings. This zero-shot semantic matching produces a strictly ranked list of task-appropriate target classes which is much smaller than the complete set of 80(e.g., 1. "wine glass", 2. "cup", 3. "bottle").

2. Dynamic Vision Acceleration (FPGA Hardware Layer)

Prior to processing the static image, the VEGA CPU transmits this filtered target list to the Genesys-2 FPGA across the system bus.
This dynamically reprograms the vision accelerator, effectively blinding the hardware to irrelevant classes and conserving significant computational power. The static image is then passed through our custom Convolutional Neural Network (CNN) accelerator running a YOLOE Nano backbone.
Because YOLOE utilizes one-to-one bipartite matching during its offline training phase, it natively outputs single, optimal bounding boxes. This completely eliminates the need for Non-Maximum Suppression (NMS), allowing the FPGA to execute a pure, uninterrupted forward-pass matrix calculation.

3. Strict Semantic Arbitration (VEGA Post-Processing Layer)

The FPGA returns an array of raw visual detections (unfiltered bounding boxes and class labels) back to the VEGA processor. To ensure the final output strictly adheres to the task's contextual requirements, the software executes a semantic arbitration algorithm.

The CPU iterates down the original priority list generated in Phase 1. The highest-ranked item on that list that is successfully identified in the FPGA's visual detections is selected as the final target. The system then outputs this optimal bounding box and class label, successfully completing the task-driven visual pipeline.

IV. Analysis of Final Design

A. Architectural Advantages

The proposed Hardware/Software Co-Design addresses edge deployment bottlenecks through four critical optimizations:

1. NMS-Free Throughput[5]: By utilizing YOLOE's one-to-one bipartite matching during training, the network natively predicts a single optimal bounding box per object. This entirely eliminates Non-Maximum Suppression (NMS), a sequential sorting algorithm that traditionally stalls FPGA pipelines, enabling a continuous, parallelized forward-pass matrix calculation.

2. Anchor-Free Regression: The detection head abandons memory-heavy anchor templates. Instead, it directly regresses four spatial distance vectors from an object's center point, drastically reducing the required logic gates and Block RAM (BRAM).

3. Asymmetric Partitioning: The VEGA CPU handles variable-length NLP tokenization (MiniLM), while the FPGA’s DSP slices process the heavy Convolutional MAC operations. Pre-filtering the hardware’s vocabulary prevents the FPGA from wasting power computing irrelevant classes.

4. INT8 Edge Quantization: Post-training INT8 quantization reduces the vision backbone's memory footprint by ~75%. This minimizes power-hungry external DDR memory accesses, allowing the majority of data to remain within the FPGA's ultra-fast internal BRAM.

B. Possible Hardware Optimization & Feasibility Analysis

1. Strategic Convolution Acceleration: While the Winograd Algorithm significantly reduces multiplication operations for 3x3 convolutions(Which YOLOE heavily rely on[6]), we must account for transformational overhead. If Winograd proves viable during synthesis, we will deploy Matrix Multiplication Units (MMUs). Otherwise, we will utilize highly parallelized Multiply-Accumulate (MAC) elements to ensure consistent, low-latency convolution throughput.

2. Activation Function Benchmarking: We will benchmark replacing the native SiLU activation with ReLU. SiLU requires hardware-intensive Look-Up Tables (LUTs) or DSPs to compute exponentials. Conversely, ReLU utilizes a simple sign-bit multiplexer, drastically reducing BRAM usage. We will profile this to maximize throughput without unacceptable drops in detection accuracy.

3. Model Compression & Selection: The choice of YOLO-E Nano provides a highly optimized baseline for edge deployment. Given its already lean architecture, further pruning and quantization may yield diminishing returns. However, we will conduct a feasibility check to squeeze out any remaining efficiency. The inherent efficiency of this model validates our selection of a high-performance, software-optimized backbone.

4. Memory Pipeline & Data Flow: To eliminate severe DDR memory bottlenecks, we will implement row-pass access patterns coupled with a Line-Buffer Architecture. Because the FPGA’s BRAM cannot store full high-resolution images, we will stream the image sequentially in pixel rows[7], process them immediately through the convolution window, and discard them, ensuring maximum on-chip data reuse.

5. Direct Text-Embedding Injection: Instead of translating text vectors back into discrete class IDs on the CPU, we plan to transmit the raw 384-dimensional text embeddings directly to the FPGA via the system bus. This enables true dynamic feature-level fusion within the vision model, preserving semantic nuance and eliminating CPU-side translation overhead.

V. Testing and Implementation of Final Design

The test methodology evaluates a prompt-guided detection pipeline where each text query is embedded, matched against all 80 COCO classes using cosine similarity, and then processed in ranked batches of 10 classes through YOLOE. For each batch, the model is re-parameterized and run once; the process stops early as soon as detections appear, and the best result is selected using a combined semantic-rank and confidence score. The benchmark records per-batch re-parameterization and inference time, end-to-end latency, RAM/CPU usage, detection count, and classes skipped due to early exit. In current runs, detections are usually found within 1-3 batches, reducing compute versus full-scan evaluation while still returning usable results.

Fig 2: Sample Test Results

V. Results and Discussion[9][10]

```
Prompt:   'Something to eat'
Image:    'D:\DVCon\YOLO-World\000000001268.jpg'
Strategy: batches of 10 · max 8 re-params

  Batch    Reparam    Infer      RAM   CPU%  Dets  Status
  ──────────────────────────────────────────────────────────
    1    1473.6ms   158.0ms   706.2MB  654.3     0  no match
    2     512.2ms    86.7ms   726.0MB  867.0     1  FOUND — stopping

  Early exit at batch 2/8 — skipped 6 batches (60 classes unscanned)
  Wall time: 2608.0ms  (2 re-params instead of 8)

==================================================
Best: BIRD
  Score: 0.0188  Conf: 0.263  Rank: #14/80
  BBox: [245 226 266 252]
==================================================
```

```
Prompt:   'Something drink tea from'
Image:    'D:\DVCon\DVCcon-SittingDucks\YOLOE\images\000000001268.jpg'
Strategy: batches of 10 · max 8 re-params

  Batch    Reparam    Infer      RAM   CPU%  Dets  Status
  ──────────────────────────────────────────────────────────
    1    1849.6ms   187.2ms   711.7MB  619.5     0  no match
    2     633.8ms   108.8ms   746.5MB  654.9     1  FOUND — stopping

  Early exit at batch 2/8 — skipped 6 batches (60 classes unscanned)
  Wall time: 3193.0ms  (2 re-params instead of 8)

==================================================
Best: BIRD
  Score: 0.0131  Conf: 0.263  Rank: #20/80
  BBox: [245 226 266 252]
==================================================
```

Fig 3: Sample Test Benchmarks

Testing validated the efficiency of the proposed semantic pre-filtering pipeline. By processing the MiniLM-ranked vocabulary in batches of 10, the system consistently triggered an early exit by the second batch. This skipped 75% of the visual search space (60 unscanned classes), proving the asymmetric architecture drastically reduces computational workload. Visual inference times were highly efficient, dropping to 86.7ms, while maintaining a stable memory footprint under 750MB, successfully avoiding the memory bloat typical of monolithic Vision-Language Models.

There is an opportunity to consolidate the semantic pre-ranking step by using CLIP text embeddings directly, eliminating the separate MiniLM model and reducing redundant encoding overhead.

VI. Conclusion

This project introduces a novel integration of YOLOE and MiniLM to address the problem statement, a combination currently unexplored in given literature. By offloading computationally intensive tasks to a custom FPGA-based hardware accelerator, we aim to deliver a high-throughput, low-power solution suitable for edge deployment. Initial software-only

testing confirms strong accuracy and feasibility, providing a robust foundation for hardware acceleration.

VII. References

[1] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, "YOLO-World: Real-Time Open-Vocabulary Object Detection," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[2] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," *European Conference on Computer Vision (ECCV)*, pp. 213-229, Springer, 2020.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982-3992, 2019.

[4] S. Xu et al., "PP-YOLOE: An evolved version of YOLO," *arXiv preprint arXiv:2203.16250*, 2022.

[5] R. Sapkota et al., "YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection," arXiv preprint arXiv:2509.25164, 2025.

[6] A. Wang et al., "YOLOE: Real-Time Seeing Anything," arXiv preprint arXiv:2501.12154, 2025. [8, 27]

[7] D. T. Nguyen et al., "A High-Throughput and Power-Efficient FPGA Implementation of YOLO CNN for Object Detection," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 27, no. 8, pp. 1861-1873, 2019. [2, 6]

[8] Y. Bian et al., "A Review of FPGA Accelerated Computing Methods for YOLO Models," 2024 4th International Conference on Internet of Things and Machine Learning (IoTML), pp. 450-457, 2024. [29, 34**]**

[9]"SittingDucks," GitHub repository. [Online]. Available: https://github.com/BimsaraU/DVCcon-SittingDucks

[10] "SittingDucks," YouTube. [Online Video]. Available: https://www.youtube.com/watch?v=ahjDf6IUxjE